DECA Lab Logbook

Part 1 | Section 1: Logic Design & Multiplexers

Section 1: Logic Design & Multiplexers

Aim

To design, build and simulate combinational logic circuits in Issie, starting from basic gates and progressing to complex hierarchical systems. The section covers AND gates, multiplexers with bus signals, and culminates in a simple bitwise ALU.

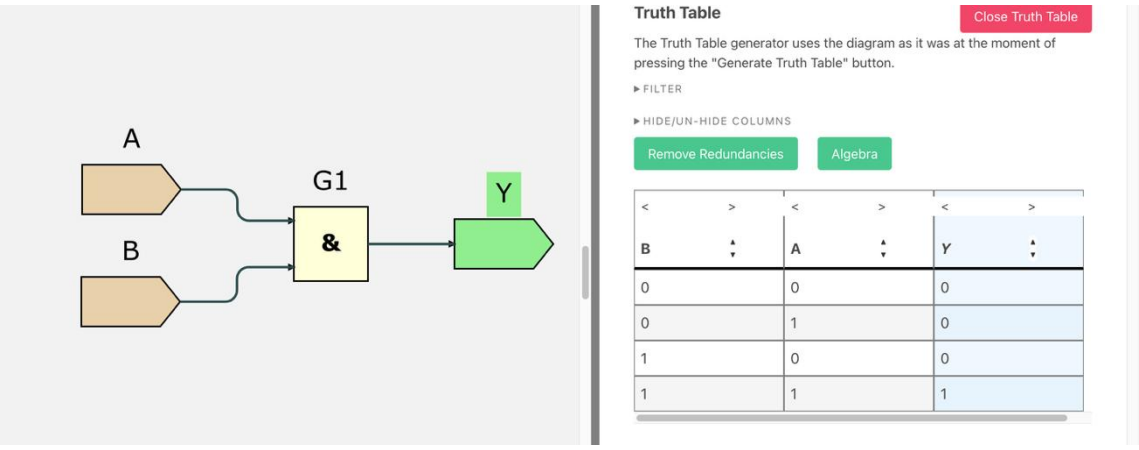
Preparation

Issie (Interactive Schematic Simulator and Integrated Editor) is used to build and simulate all logic systems. In Issie, logic gates follow the IEC rectangular-symbol standard.

Two 1-bit Input AND Gate

Circuit Description

A 2-input AND gate with inputs A and B and output Y. The Boolean function is Y = A · B.



Truth Table

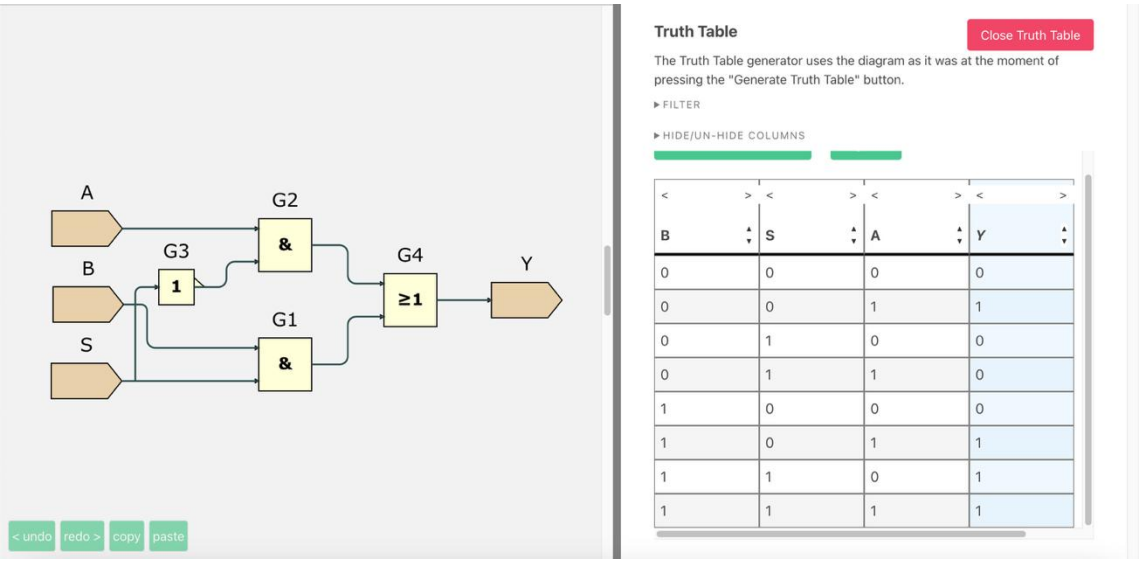
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

The truth table confirms Y = 1 if and only if both A = 1 and B = 1. Verified by exhaustive simulation in Issie.

Two 1-bit Input Multiplexer (MUX2)

Circuit Description

A 2-input multiplexer constructed from primitive gates, implementing the Boolean function Y = S̄A + SB. When S = 0, the upper AND gate passes A and the lower outputs 0, giving Y = A. When S = 1, the lower AND gate passes B and the upper outputs 0, giving Y = B.



B	S	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Components

- Three 1-bit inputs: A (data), B (data), S (select)
- One NOT gate, two AND gates, one OR gate

Truth Table (8 combinations, 2^3)

S	A	Y
0	A	A (passes input A)
1	B	B (passes input B)

Hierarchical Design

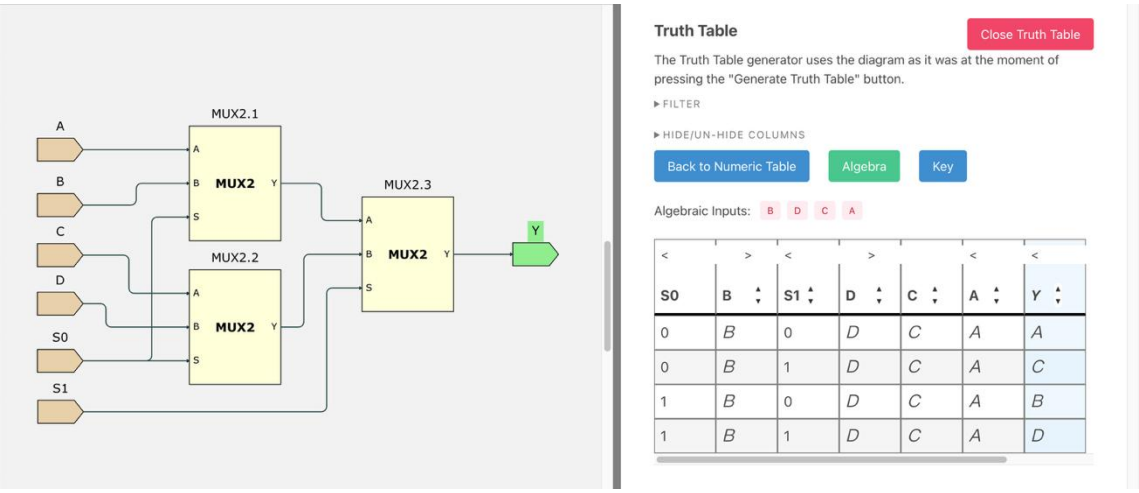
Complex systems are broken into separate sheets, each defining a reusable component. A component can be instantiated multiple times in other sheets without re-verifying its function. This is used throughout the rest of the section.

Four 1-bit Input Multiplexer (MUX4)

Circuit Description

A 4-input multiplexer using three instances of MUX2 in a tree structure. Two MUX2 instances first select between {A, B} and {C, D} using S0. A third MUX2 then selects between those two results using S1. The Boolean function is:

$$Y = A \cdot \bar{S_0} \cdot \bar{S_1} + B \cdot S_0 \cdot \bar{S_1} + C \cdot \bar{S_0} \cdot S_1 + D \cdot S_0 \cdot S_1$$



Algebraic Truth Table

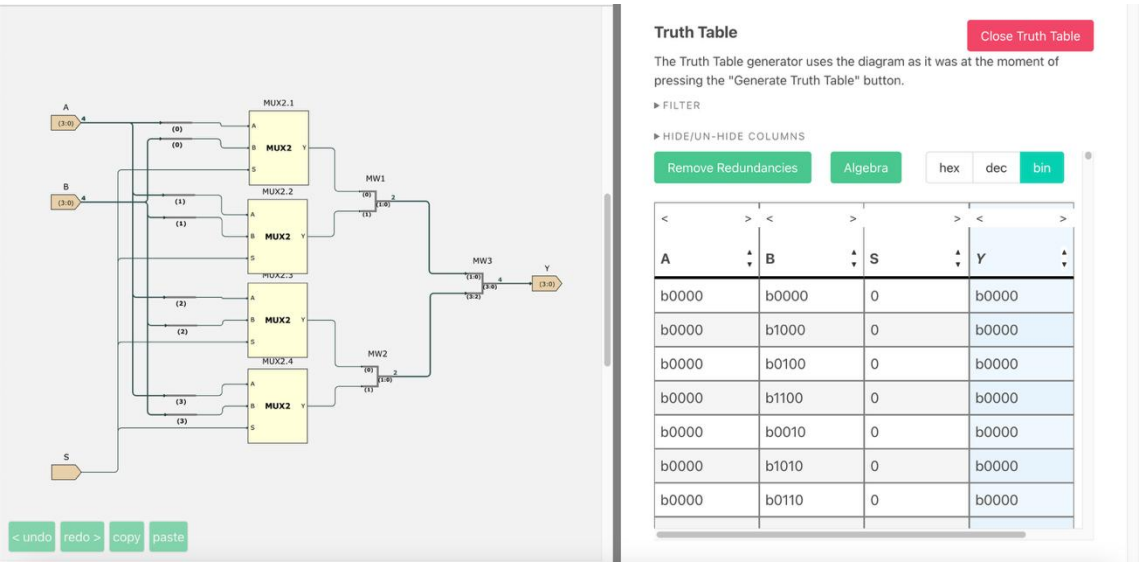
S1	S0	Y
0	0	A
0	1	B
1	0	C
1	1	D

Since there are 6 inputs (A, B, C, D, S0, S1), exhaustive testing requires $2^6 = 64$ combinations. The algebraic truth table (with A, B, C, D set to symbolic in Issie) confirms the correct input is selected for each combination of S.

Two 4-bit Input Multiplexer (MUX2X4)

Circuit Description

The 1-bit MUX2 is extended to operate on 4-bit buses. Four MUX2 instances each handle one bit. BusSelect components extract individual bits from the 4-bit input ports A(3:0) and B(3:0). MergeWires recombines the four 1-bit outputs into the 4-bit output Y(3:0). The single select line S controls all four MUX2 instances together.



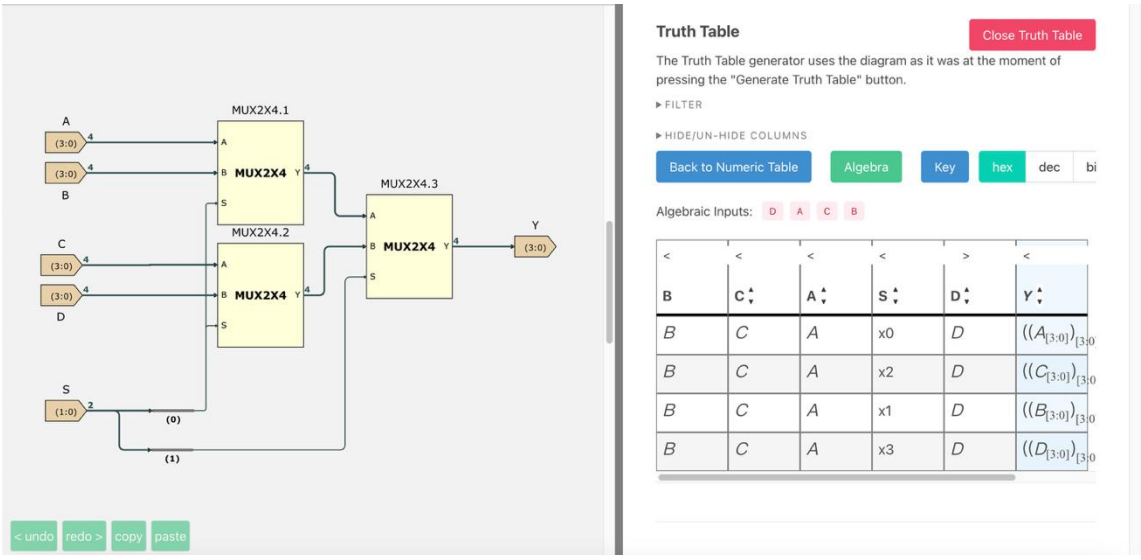
Test Results

A (4-bit)	B (4-bit)	S	Y
0001	1110	0	0001
0001	1110	1	1110
0010	1101	0	0010
0010	1101	1	1101
0100	1011	0	0100
0100	1011	1	1011
1000	0111	0	1000
1000	0111	1	0111

Four-input 4-bit Multiplexer (MUX4X4)

Circuit Description

Three instances of MUX2X4 are arranged in the same tree structure as MUX4, but all data paths are now 4-bit buses. Individual bits of the 2-bit select bus S(1:0) are extracted to control each layer of the tree. No bus splitting or merging is needed for the data paths since all port widths match.



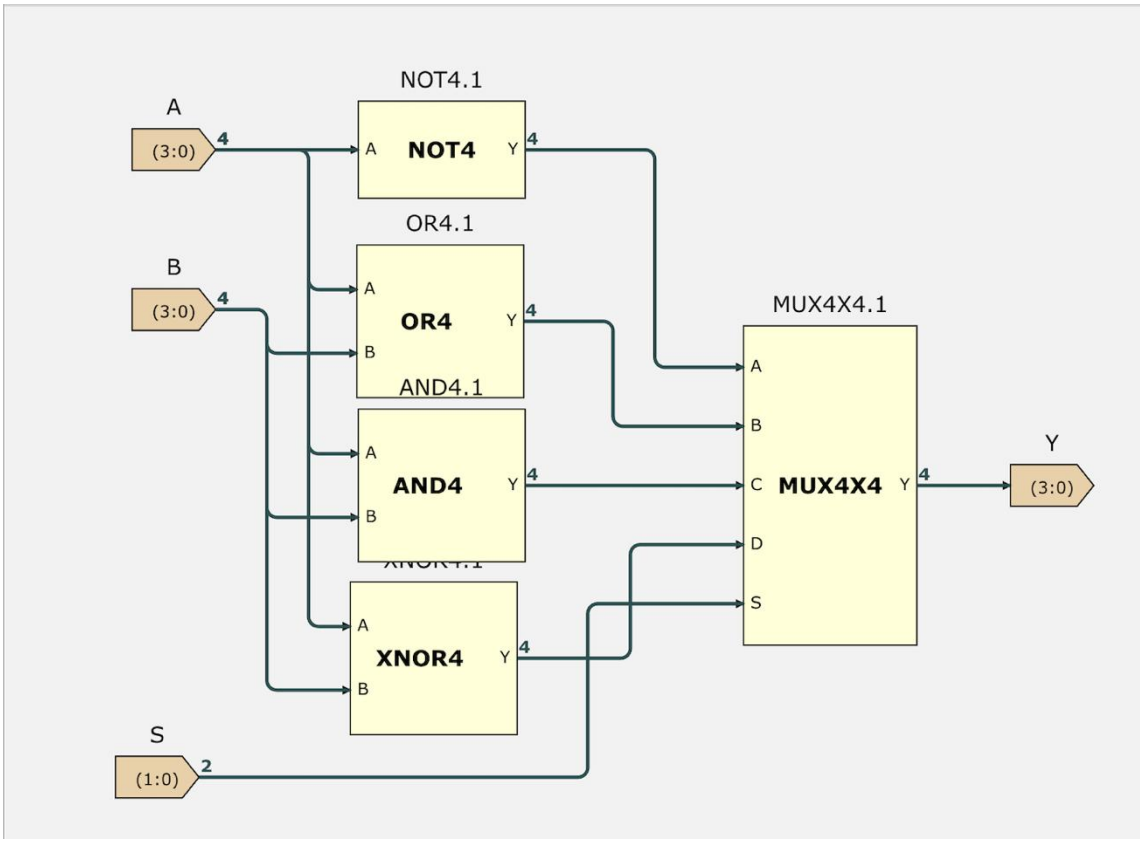
Challenge: Multi-purpose Logic Gate (Bitwise ALU)

Aim

Build a 4-bit ALU that supports four bitwise operations, selected by a 2-bit instruction code S. The four operations are computed in parallel and a MUX4X4 selects the result to output.

Components

- NOT4: four NOT gates operating on each bit of A
- OR4: four OR gates performing A OR B bitwise
- AND4: four AND gates performing A AND B bitwise
- XOR4: four XOR gates performing A XOR B bitwise
- MUX4X4: selects which operation result to output based on S



Instruction Table

S (binary)	Operation	Y
b00	Bitwise NOT A	$\bar{A}$
b01	A OR B	$A + B$
b10	A AND B	$A \cdot B$
b11	A XOR B	$A \oplus B$

Verified using Issie's algebraic truth table: for each value of S, Y correctly reflects the corresponding bitwise operation on A and B.